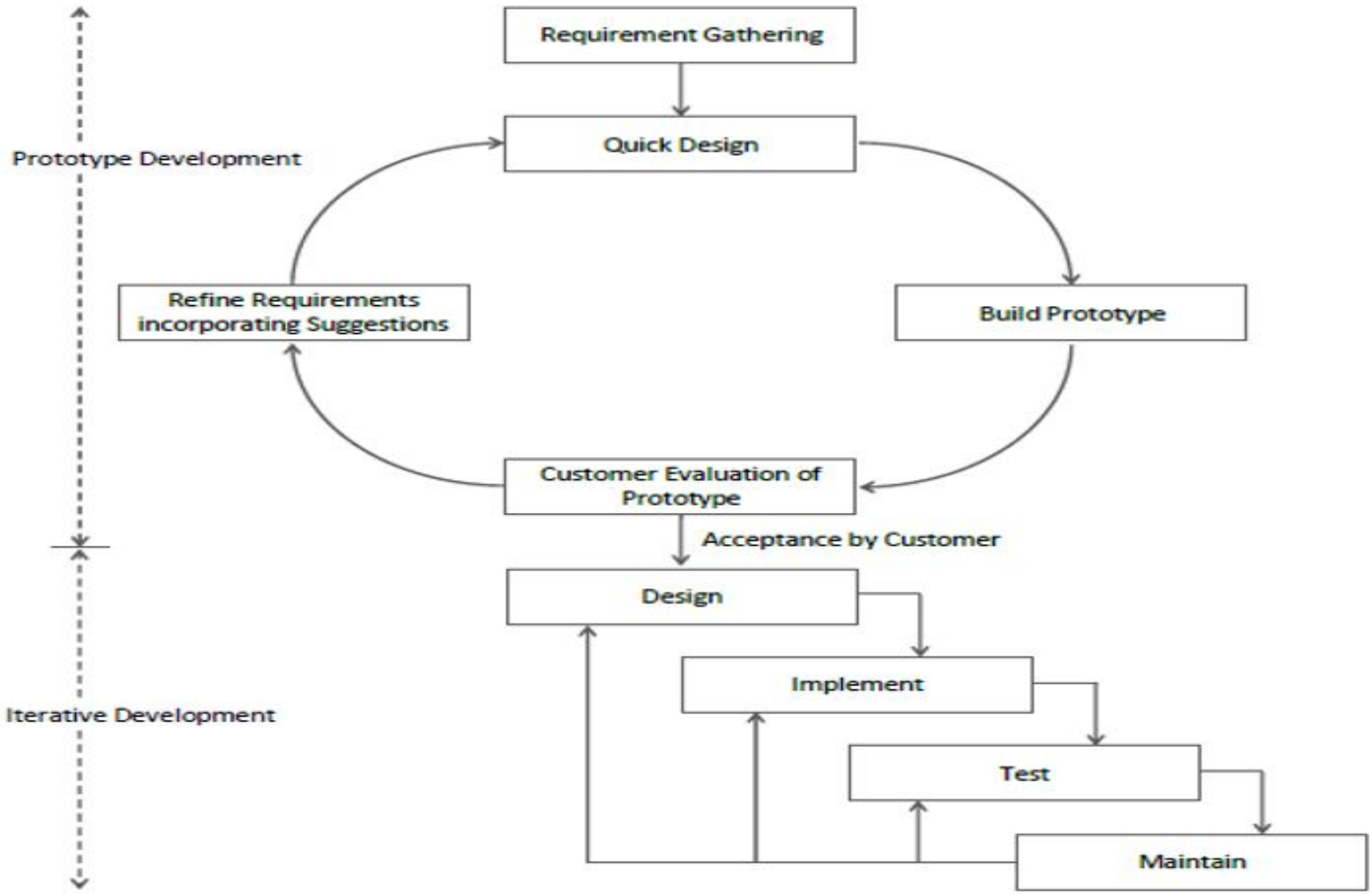
Software Engineering

Lecture six

THE PROTOTYPING MODEL

The prototyping paradigm begins with requirements gathering. Developer and customer meet and define the overall objectives for the software, identify whatever requirements are known, and outline areas where further definition is mandatory. A "quick design" then occurs. The quick design focuses on a representation of those aspects of the software that will be visible to the customer/user The quick design leads to the construction of a prototype. The prototype is evaluated by the customer/user and used to refine requirements for the software to be developed. Iteration occurs as the prototype is tuned to satisfy the needs of the customer, while at the same time enabling the developer to better understand what needs to be done.

The prototyping paradigm



Stages of the Prototyping Model

1. Requirement Gathering

Only the basic requirements are collected to start building the initial prototype. Detailed requirements evolve as feedback is received.

2. Quick Design

A rough design is created for the prototype, focusing only on visible features and user interfaces, not the full architecture.

3. Prototype Development

The prototype is built using minimal functionality to simulate the expected behavior of the actual software.

4. User Evaluation

The client or end-users interact with the prototype and provide feedback on design, features, usability, etc.

5. Refinement

Based on user feedback, the prototype has been improved and tested again. This loop continues until the client is satisfied.

6. Product Development

Once the prototype is approved, the final prouct is developed from scratch using the clarified and validated requirements.

Advantages of the Prototype Model

Improved User Involvement: Clients see early versions and help shape the final product.

Better Requirement Clarity: Reduces misunderstandings and incomplete requirements.

Early Problem Detection: Design and usability issues are caught early.

Cost-effective Changes: Easier and cheaper to implement changes in the early stages.

❌ Disadvantages of the Prototype Model

Increased Time and Cost: Building multiple prototypes may increase overall effort.

Scope Creep: Users may demand more features than originally planned.

Neglected Quality: Quick designs may overlook technical standards and performance.

Misleading Functionalities: Users may assume the prototype is the final product, confusing.

EVOLUTIONARY SOFTWARE PROCESS MODELS

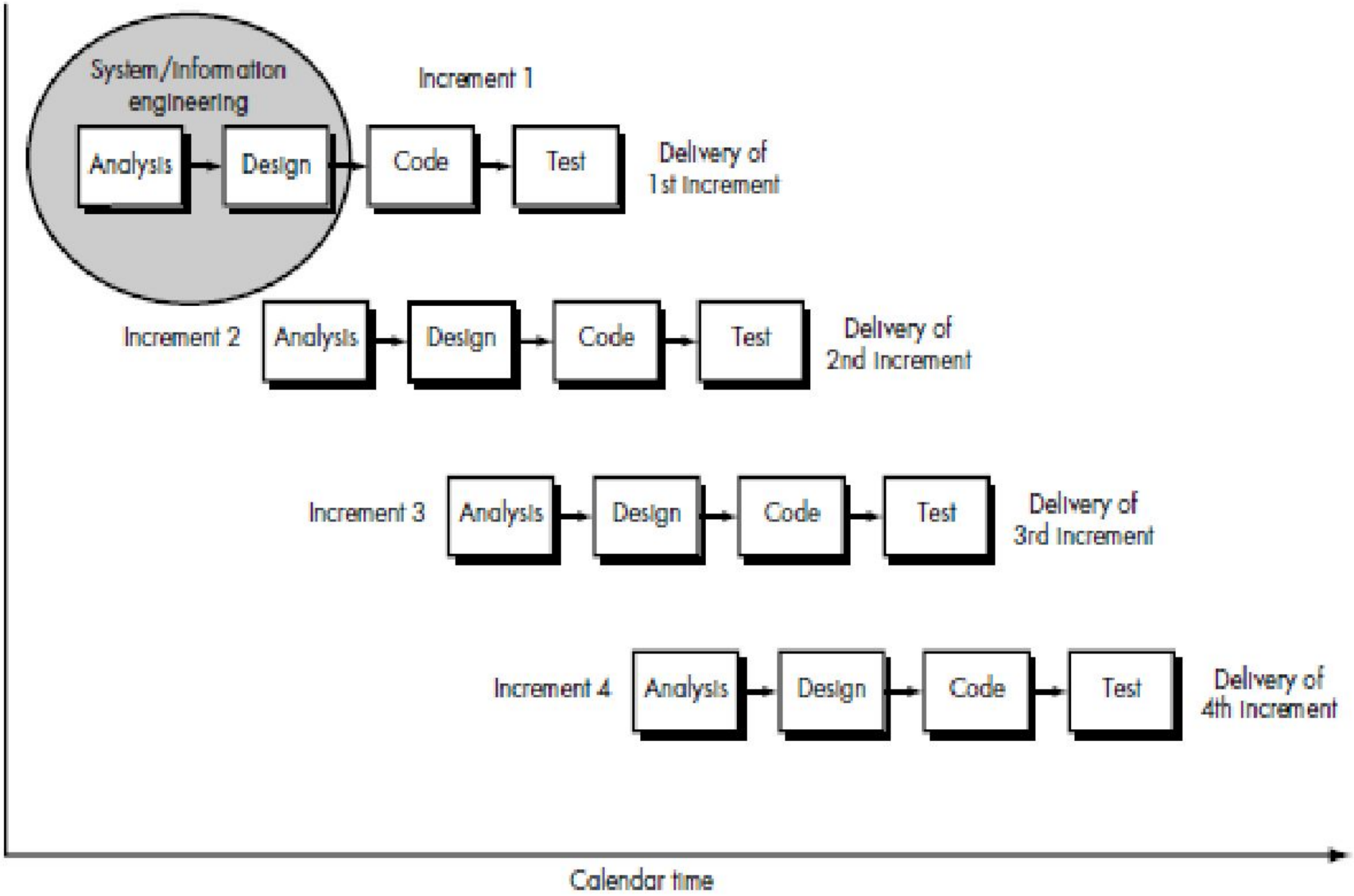
Evolutionary models are iterative. They are characterized in a manner that enables software engineers to develop increasingly more complete versions of the software.

1- **The Incremental Model**

The incremental model combines elements of the linear sequential model with the iterative philosophy of prototyping.

Incremental development is particularly useful when staffing is unavailable for a complete implementation by the business deadline that has been established for the project. Early increments can be implemented with fewer people. In addition, increments can be planned to manage technical risks.

The Incremental Model



Advantages of Incremental model:

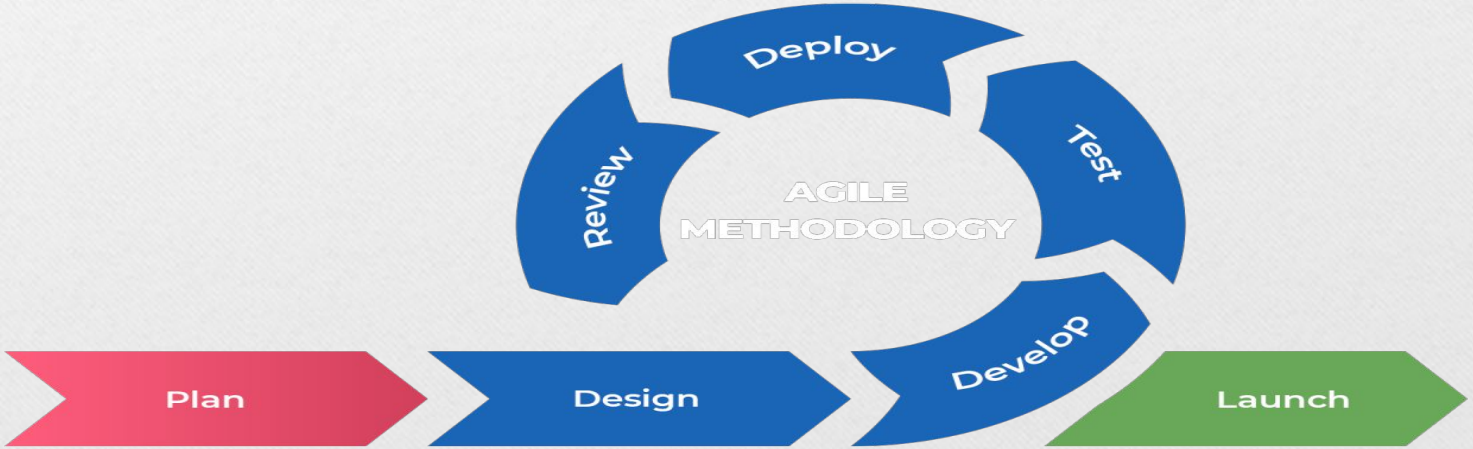
- Generates working software quickly and early during the software life cycle.
- This model is more flexible – less costly to change scope and requirements.
- It is easier to test and debug during a smaller iteration.
- In this model customer can respond to each built.
- Lowers initial delivery cost.
- Easier to manage risk because risky pieces are identified and handled during it'd iteration.

Disadvantages of Incremental model:

- Needs good planning and design.
- Needs a clear and complete definition of the whole system before it can be broken down and built incrementally.
- Total cost is higher than waterfall.

2-The Agile model

The Agile model in software engineering is an iterative and incremental approach that focuses on flexibility, continuous delivery, and customer collaboration to produce high-quality software quickly. It breaks projects into small, manageable cycles (iterations) that allow teams to adapt to changing requirements promptly, rather than relying on strict, upfront planning



Key Aspects of Agile Model:

- **Iterative & Incremental**: Software is developed in small, functional releases (iterations).
- **Customer Collaboration**: Active customer involvement ensures the product meets evolving needs.
- **Flexibility**: Changes are welcome at any stage, even in late development.
- **Core Principles**: Based on the Agile Manifesto, prioritizing individuals, working software, customer collaboration, and responding to change.

- **Agile Model Development Lifecycle:**
- **Concept**: Define the project scope and initial requirements.
- **Inception**: Build the team, create a backlog, and outline requirements.
- **Iteration**: Develop, code, and unit test functionality based on feedback.
- **Release**: Perform testing, training, and final deployment.
- **Maintenance:** Provide ongoing support and updates.
- **Retirement**: End-of-life, migrating to new versions.

Advantages of Agile Model:

Rapid Delivery: Frequent releases ensure faster time-to-market.

High Quality: Continuous testing and feedback loops improve the final product.

Customer Satisfaction: Regular demonstration of working software ensures alignment with customer needs.

Reduced Risks: Early identification of issues allows for immediate, corrective actions.